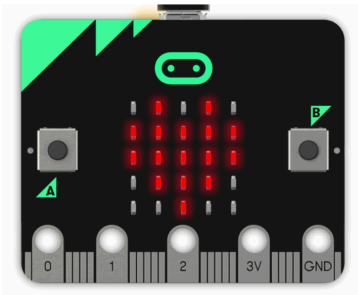
Lab 3: Interrupts



Hilary Term 2026

Mark van der Wilk — Lecturer & Adaptations
Mike Spivey — Lecturer & Course Designer

Contents

1. Introduction 1
2. Tasks 2

ⓘ **Much to read, not as much to do. Read *before* the scheduled lab session!**

1. Introduction

This lab begins with a pair of programs that output a list of primes on the serial port. One of the programs uses polling to wait for the serial port to be ready before transmitting each character; the other buffers the characters waiting to be output, and uses interrupts to send each character when the port is ready.

The lab3-primes directory of the lab materials contains the following files:

File Name	Description
`Makefile`	Build script
`primes-poll.c`	Main program that uses polling
`primes-intr.c`	Main program that uses interrupts
`hardware.h`	Header file with layout of I/O registers
`lib.c, lib.h`	Library with implementation of printf
`startup.c`	Startup code
`nRF51822.ld`	Linker script
`lab3.geany`	Geany project file

Typing `make` as usual (or selecting Build>Make in Geany) will build two version of the primes program: in `primes-poll.hex` is the version that uses polling, and in `primes-intr.hex` is the interrupt-driven version. The implementation of the function `serial_putc` that does the work of `printf` is different in the two programs, and the interrupt-driven program has an additional function with the special name `uart_handler` that the hardware calls when a UART interrupt is triggered.

One of the LEDs on the micro:bit is turned on while the program is running and printing the first 500 primes, and turned off at the end. You can time the program with a watch, or wire the board up to an oscilloscope or logic analyser to get a more accurate timing.

2. Tasks

Requirements for S

1. Modify `primes-poll.c` so that transmission of each character completes before `serial_putc` returns, rather than before transmitting the character on the next call. Does this have a measurable effect on the running time?
2. How small can you make the transmit buffer and still have the interrupt-driven version `primes-intr.c` work? Does a very small buffer adversely affect the running time?
3. Add code to monitor the maximum number of characters stored in the transmit buffer, and print it at the end. Try increasing the buffer size to larger powers of two – you should be able to use values up to 8192 – and see if the whole buffer is ever filled.
4. What happens to the running time of both programs if we modify them to print not the first 500 primes, but the first 500 primes that are more than 1 000 000 or 10 000 000?

Requirements for S+

- If the calls to `intr_disable` and `intr_enable` in `serial_putc` are removed, does the program continue to work? Can you persuade it to go wrong? What if the critical section is reduced to cover only the `else` part of the conditional `if (txidle)`?

OR

- Find out from Chapter 21 of the hardware reference manual for the nRF51822 how to configure the random number generator. Write a driver based on the hints given in Problem Sheet 3, and write a program that generates and prints a histogram showing the distribution of random values. Note: symbolic constants for the device addresses of the RNG are in the latest revision of the source file `hardware.h`.

Further practice:

- Add code to turn on an LED when the program is searching for the next prime, and turn it off when it is printing a prime it has found. Use a scope or logic analyser on the LED and the serial line to visualise the overlap between thinking and printing. TODO: Led should be on when transmitting.
- When an interrupt occurs, the register values are saved on the stack. This ought not to affect the functioning of properly written code that has been correctly translated: but can you write some sneaky code to detect that memory just beyond the top of the stack is changing in an unpredictable way? Hint: the loop in `serial_putc` ought to experience some interrupts.
- A bizarre challenge: find out if serial transmission can be implemented by bit-banging. This will mean configuring the correct pin as a GPIO output, and using delay loops to generate the RS-232 waveform with the correct timing. Use of an oscilloscope or logic analyser will be essential to get the timing right.